

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Avinash K	Reg. No.:	192411044
Section / Batch:	CSE	Date:	3/8/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Evaluate the efficiency of Quick Sort using different pivot selection strategies such as first element, last element, and median-of-three. Execute the algorithm on random, sorted, and reverse-sorted inputs. Record execution time and compare the results. Interpret how pivot selection influences performance and justify the best strategy for avoiding worst-case scenarios.

Code of Conduct

I Avinash k(192411044) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: AvinashK

```

import random
import time
import sys
import copy

sys.setrecursionlimit(20000)

def quicksort(arr, low, high, pivot_strategy, counters):
    if low < high:
        pivot_index = partition(arr, low, high, pivot_strategy, counters)
        quicksort(arr, low, pivot_index - 1, pivot_strategy, counters)
        quicksort(arr, pivot_index + 1, high, pivot_strategy, counters)

def choose_pivot(arr, low, high, pivot_strategy):
    if pivot_strategy == "first":
        return low
    elif pivot_strategy == "last":
        return high
    elif pivot_strategy == "median3":
        mid = (low + high) // 2
        a, b, c = arr[low], arr[mid], arr[high]
        if a <= b <= c or c <= b <= a:
            return mid
        elif b <= a <= c or c <= a <= b:
            return low
        else:
            return high
    else:
        raise ValueError("Unknown pivot strategy")

def partition(arr, low, high, pivot_strategy, counters):
    pivot_idx = choose_pivot(arr, low, high, pivot_strategy)
    arr[pivot_idx], arr[high] = arr[high], arr[pivot_idx]
    pivot = arr[high]

    i = low - 1
    for j in range(low, high):
        counters["comparisons"] += 1
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
            counters["swaps"] += 1
    arr[i + 1], arr[high] = arr[high], arr[i + 1]

```

```
counters["swaps"] += 1
return i + 1
```

```
def run_trial(data, pivot_strategy):
    arr = copy.deepcopy(data)
    counters = {"comparisons": 0, "swaps": 0}
    start = time.perf_counter()
    quicksort(arr, 0, len(arr) - 1, pivot_strategy, counters)
    end = time.perf_counter()
    assert arr == sorted(data), "Sort failed!"
    return {
        "time_sec": end - start,
        "comparisons": counters["comparisons"],
        "swaps": counters["swaps"],
    }
```

```
def make_datasets(n):
    random_data = [random.randint(0, 1_000_000) for _ in range(n)]
    sorted_data = list(range(n))
    reverse_data = list(range(n, 0, -1))
    return {
        "Random": random_data,
        "Sorted": sorted_data,
        "Reverse-sorted": reverse_data,
    }
```

```
def main():
    n = 2000 # array size (kept moderate so first/last-pivot worst cases finish quickly)
    strategies = ["first", "last", "median3"]
    datasets = make_datasets(n)

    results = {}
    for dataset_name, data in datasets.items():
        results[dataset_name] = {}
        for strategy in strategies:
            results[dataset_name][strategy] = run_trial(data, strategy)

    # Print results table
    print(f'Array size (n) = {n}\n')
    header = f'{"Input Type":<16} {"Pivot Strategy":<16} {"Time (s)":<12} {"Comparisons":<14} {"Swaps":<10}'
    print(header)
    print("-" * len(header))
    for dataset_name in datasets:
```

for strategy in strategies:

```
r = results[dataset_name][strategy]
```

```
print(f"{dataset_name:<16} {strategy:<16} {r['time_sec']:<12.6f}"
```

```
      f"{r['comparisons']:<14} {r['swaps']:<10}")
```

```
print()
```

return results

if __name__ == "__main__":

main()

ASSESSMENT-CO3/AT1-EXPERIMENTS

Language: Python C C++ Java

QUESTIONS

Q1

Quick Sort

10 marks

1/1 answered

Q1 Quick Sort

10 marks

Evaluate the efficiency of Quick Sort using different pivot selection strategies such as first element, last element, and median-of-three. Execute the algorithm on random, sorted, and reverse-sorted inputs. Record execution time and compare the results. Interpret how pivot selection influences performance and justify the best strategy for avoiding worst-case scenarios.

Your Code

```
1 import random
2 import time
3 import sys
4 import copy
5 sys.setrecursionlimit(20000)
6 def quicksort(arr, low, high, pivot_strategy, counters):
7     if low < high:
8         pivot_index = partition(arr, low, high, pivot_strategy, counters)
9         quicksort(arr, low, pivot_index - 1, pivot_strategy, counters)
10        quicksort(arr, pivot_index + 1, high, pivot_strategy, counters)
11 def choose_pivot(arr, low, high, pivot_strategy):
12     if pivot_strategy == "first":
13         return low
14     elif pivot_strategy == "last":
15         return high
16     elif pivot_strategy == "median-of-three":
```

Input

stdin input

Output

Array size (n) = 2000

Input Type	Pivot Strategy	
Time (s)	Comparisons	Swaps

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____